

Московский государственный университет
имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Отчет по задаче 2
Доработка MPI-программы

Вариант 20

Выполнил:
Студент гр. 421
Астраханцев Дмитрий Андреевич

Москва, 2024

Содержание

1	Исходная задача	2
2	Постановка задачи	4
3	Сделанные модификации алгоритма	5
4	Временная оценка	5

1 Исходная задача

Дан алгоритм, который производит релаксацию 3-хмерной матрицы. Каждая ячейка на новой итерации релаксации вычисляется, как среднее арифметическое 6 ячеек, которые ее окружают. Т.е.

```
1 A[i][j][k] = (A[i-1][j][k] + A[i+1][j][k] + A[i][j-1][k] + A[i][j+1][k] +  
A[i][j][k-1] + A[i][j][k+1]) / 6.}
```

Для решения данной задачи было решено разбивать исходный куб размера $N \times N \times N$ на более мелкие части размера $m \times m \times m$, где m подбирается в зависимости от размера исходных данных.

Каждый MPI-процесс получает один или несколько кубов для обработки, после этого передает граничные элементы в следующий процесс, который обрабатывает соседний куб. Отдельно стоит рассмотреть граничные элементы, которые, вообще говоря, могут иметь форму отличную от куба, если m не является делителем N .

Приведем поясняющие схемы. На схеме 1 изображен куб для исходной задачи без использования MPI. На схеме разными цветами выделены диагональные плоскости куба. Заметим, что вычисление элементов такой плоскости невозможно без вычисления элементов предыдущей плоскости.

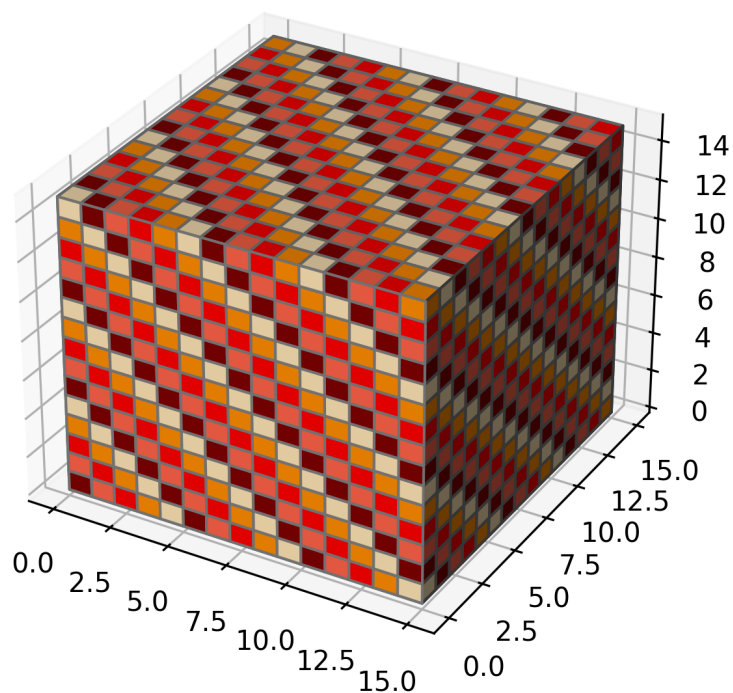


Рис. 1: Схема работы алгоритма без использования MPI

Схема 2 иллюстрирует работу алгоритма с использованием MPI. Куб был разбит на подкубы, и каждый малый куб будет вычисляться внутри отдельного MPI-процесса. Каждый процесс может брать для вычисления несколько независимых кубов.

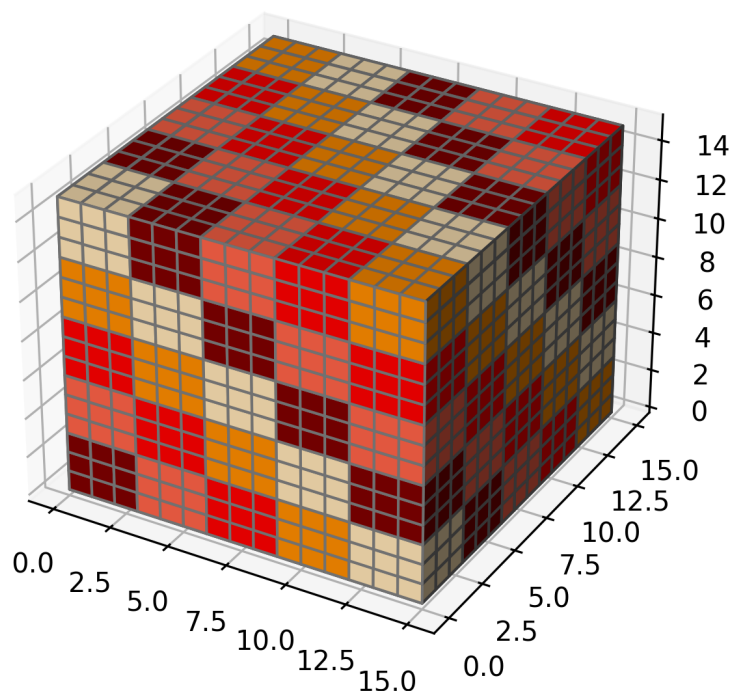


Рис. 2: Схема работы алгоритма с использованием MPI

2 Постановка задачи

Доработать MPI-программу, реализованную в рамках курса «Суперкомпьютеры и параллельная обработка данных». Используя параллельный ввод-вывод (MPI-IO), добавить контрольные точки для продолжения работы программы в случае сбоя. Реализовать один из 3-х сценариев работы после сбоя: а) продолжить работу программы только на «исправных» процессах; б) вместо процессов, вышедших из строя, создать новые MPI-процессы, которые необходимо использовать для продолжения расчетов; в) при запуске программы на счет сразу запустить некоторое дополнительное количество MPI-процессов, которые использовать в случае сбоя.

Для реализации был выбран сценарий В – при запуске программы сразу запустить некоторое дополнительное количество MPI-процессов, которые необходимо использовать в случае сбоя.

3 Сделанные модификации алгоритма

Были произведены следующие модификации алгоритма:

1. После каждой итерации релаксации добавлены контрольные точки. Каждый процесс сохраняет кубы, с которыми он работает в отдельный файл с названием `<номер процесса>.mpi`.
2. Установлена функция-`errorhandler`. При поломке процесса и попытке взаимодействовать с ним управление передается в нее. После этого сразу вызываются функции `MPIX_Comm_revoke` и `MPIX_Comm_shrink`. Первая отменяет все пересылки на коммуникаторе, а вторая изменяет номера процессов так, чтобы устранить пробел в нумерации процессов. После необходимо отменить все, что было вычислено в последней итерации, которая не дошла до конца, чтобы синхронизировать данные всех процессов. На языке C пришлось бы использовать множественное `goto`, однако в языке C++ существует механизм исключений, который очень удобно использовать для этих целей.
3. После вызова функции-`errorhandler`'а управление будет передано в `catch`-блок, в котором
 - (a) Будет произведена очистка кубов, которые использовались процессом ранее, чтобы избежать утечек памяти.
 - (b) Счетчик итераций будет уменьшен на 1, чтобы откатить последнюю итерацию.
 - (c) Будет загружена контрольная точка, соответствующая новому номеру процесса.

4 Временная оценка

Основное отличие от алгоритма, реализованного в курсе «Суперкомпьютеры и параллельная обработка данных», заключается в создании и поддержке контрольных точек. Заметим, что по сути на каждой итерации алгоритма мы сохраняем всю вычисленную матрицу. Разумеется это очень сильно замедляет программу. Таким образом можно заключить, что такая стратегия контрольных точек будет эффективна в следующих случаях:

1. Одна итерация является достаточно «дорогой» (по времени или вычислительным ресурсам) и нам действительно важно сохранять прогресс после каждой итерации.
2. Запись контрольной точки является относительно «дешевой операцией» и не будет сильно увеличивать время работы программы.